

A Low-Code Framework for Conversational AI Assistants Leveraging Retrieval-Augmented Generation and LLMs

¹Thanigaivel G

Department of Computer Science
and Engineering,
Vels Institute of Science
Technology and Advanced
Studies(VISTAS)
Chennai, India
thanigaivelrubesh1@gmail.com

²Thirumal S

Department of Computer Science
and Engineering,
Vels Institute of Science
Technology and Advanced
Studies(VISTAS)
Chennai, India
thirumal.se@vistas.ac.in

³Kumar N

Department of Computer Science
and Engineering,
Vels Institute of Science
Technology and Advanced
Studies(VISTAS)
Chennai, India
kumar.se@vistas.ac.in

Abstract — *Conversational AI has evolved rapidly with the advent of Large Language Models (LLMs), enabling intelligent, context-aware interactions between users and systems across various domains. These models have demonstrated remarkable capabilities in understanding natural language and generating human-like responses. However, integrating such advanced AI into enterprise applications remains a challenge, often requiring significant programming effort, AI expertise, and system customization. To address this gap, this project presents a low-code framework for building a domainspecific Conversational AI Assistant using Retrieval-Augmented Generation (RAG) and LLMs. The framework is designed to integrate seamlessly with platforms like Oracle APEX, enabling developers and business users to create intelligent assistants with minimal technical complexity. By combining the generative power of LLMs with the precision of retrieval-based methods, the assistant can deliver accurate, real-time responses sourced from enterprise knowledge bases, including documents, databases, and APIs.*

Keywords — *Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Low-code development, Oracle APEX, Conversational AI*

I. INTRODUCTION

The rapid evolution of Conversational Artificial Intelligence (AI) has transformed how humans interact with digital systems, offering intelligent, context-aware, and natural communication across diverse domains such as enterprise management, customer service, and education. With the advent of

Large Language Models (LLMs), conversational agents have gained remarkable capabilities in understanding and generating human-like text. However, despite their potential, integrating LLM-powered assistants into enterprise workflows remains a challenging task due to the need for extensive coding, technical expertise, and complex system integration.

Traditional chatbots, often rule-based or template-driven, struggle with dynamic knowledge retrieval and lack contextual understanding, making them unsuitable for enterprise environments that require domain-specific, real-time, and accurate responses. To address these limitations, Retrieval-Augmented Generation (RAG) has emerged as a promising approach by enhancing LLMs with access to structured and unstructured knowledge sources. This combination allows chatbots to retrieve domain-relevant information while maintaining fluent and context-rich conversations.

This work proposes a low-code framework for building enterprise-ready Conversational AI Assistants that leverage RAG and LLMs, integrated with platforms such as Oracle APEX. By adopting a low-code paradigm, the framework minimizes development complexity, lowers technical barriers, and enables non-developers to configure and deploy

intelligent assistants seamlessly. The system architecture integrates natural language understanding, document retrieval, and response generation into a unified workflow, ensuring accuracy, scalability, and accessibility.

The proposed solution has been evaluated based on its usability, integration efficiency, and ability to provide contextually relevant responses. Results demonstrate that the low-code approach significantly reduces development effort while enhancing the effectiveness of AI-driven assistants in enterprise applications such as ERP and CRM systems.

II. RELATED WORK

Conversational AI has attracted significant attention in recent years, particularly with the rise of Large Language Models (LLMs) and their ability to generate coherent, context-aware responses. Traditional chatbots, largely based on rule-based or retrieval-only approaches, often lacked adaptability and struggled to handle domain-specific queries effectively. The integration of LLMs with knowledge retrieval techniques such as Retrieval-Augmented Generation (RAG) has addressed some of these limitations by combining generative fluency with factual accuracy.

For instance, Yin and Decker (2025) [1] demonstrated the use of LLM-driven chatbots in higher education, where domain-specific knowledge integration was shown to improve accuracy in answering database-related queries. Similarly, Sarferaz (2025) [2] explored the implementation of generative AI within ERP software, highlighting the importance of contextual understanding for business process automation.

Low-code platforms have also been leveraged to accelerate AI adoption in enterprise workflows. Tools such as Oracle APEX and Microsoft Power Platform provide simplified development environments for building AI-driven applications. Research describes

emphasized how low-code/no-code tools reduce the entry barrier for non-technical users, enabling rapid deployment of AI-powered assistants in enterprise systems. However, these tools often lack native support for integrating advanced AI models with domain-specific knowledge bases.

Recent works have also explored hybrid approaches to conversational agents. LlamaIndex (2024) [12] and LangChain (2024) [13] have provided frameworks for combining vector databases with LLMs, enabling efficient semantic retrieval for context augmentation. These frameworks demonstrate strong potential for enterprise use but typically require significant technical expertise for customization and deployment.

Other optimization-inspired architectures have focused on improving retrieval efficiency and reducing computational overhead. For example, Yung et al. (2025) [3] proposed a scalable RAG pipeline integrated with FAISS for large-scale document search, reporting significant performance improvements in real-time enterprise query handling. Likewise, OpenAI (2024) [14] introduced advancements in embedding-based retrieval that enhanced contextual grounding across diverse domains.

Building on these studies, this project proposes a low-code framework for LLM-driven conversational assistants integrated with RAG, designed to bridge the gap between enterprise usability and advanced AI capabilities. Unlike prior work that requires extensive coding or infrastructure, the proposed system emphasizes accessibility, modularity, and ease of integration with ERP and CRM systems.

III. PROPOSED METHODOLOGY

The proposed methodology introduces a low-code conversational AI framework that integrates Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG) to deliver context-

aware and domain-specific responses in enterprise environments. The framework is designed to minimize development complexity through low-code tools while ensuring scalability, accuracy, and seamless integration with organizational workflows. The following subsections describes the flow of the process.in Figure 1.

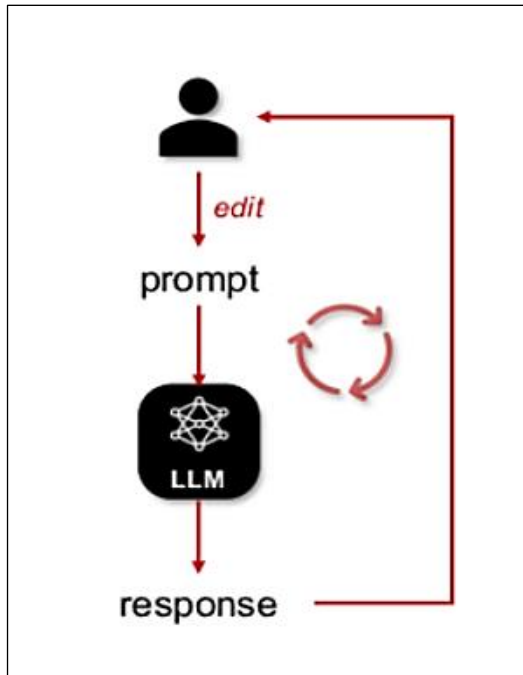


Fig. 1. Module Flow Diagram

A. USER QUERY HANDLING

The process begins when a user submits a natural language query through a chat-based interface developed using low-code platforms such as Oracle APEX. The interface is designed to be intuitive and accessible, allowing users to interact with enterprise systems without technical expertise. The queries are then passed to the middleware layer for further processing.

B. EMBEDDING GENERATION

To enable semantic understanding, the system converts user queries into high-dimensional vector representations using pretrained embedding models such as Open AI Embeddings or Sentence-BERT.

This step ensures that the intent and contextual meaning of the input are captured, enabling more effective retrieval of relevant information from enterprise knowledge bases.

C. KNOWLEDGE RETRIEVAL

The enterprise knowledge sources, including structured databases, ERP systems, and unstructured documents, are indexed into a vector database such as FAISS vector. A similarity search is performed against these stored embeddings to retrieve the most relevant documents or records that can enrich the LLM's response with factual, domain-specific knowledge.

1) DOCUMENT PREPROCESSING

The first step in RAG involves preparing enterprise knowledge sources such as ERP data, structured databases, and unstructured documents. These sources are cleaned, segmented into manageable chunks, and converted into dense vector representations using embedding models like OpenAI Embeddings or Sentence-BERT. The resulting vectors are then stored in a vector database (e.g., FAISS to enable efficient similarity-based retrieval.

2) QUERY EMBEDDING

When a user submits a query, it is encoded into a vector representation using the same embedding model applied during indexing. A similarity search is then performed against the vector database to retrieve the most relevant documents. This ensures that the assistant can access domain-specific knowledge dynamically, rather than depending on static model parameters.

3) CONTEXT INJECTION

The retrieved documents are combined with the user's query to form an augmented prompt. This prompt construction step provides the LLM with both the context of the question and the supporting evidence retrieved from enterprise data sources. As a result, the generated responses are more reliable,

factually grounded, and aligned with organizational knowledge.

D. AUGMENTED PROMPT CONSTRUCTION

Once the relevant knowledge is retrieved, it is combined with the original user query to construct an augmented prompt. This augmented prompt ensures that the LLM has access to both the user's intent and the supporting context from enterprise data, reducing hallucinations and improving factual accuracy in the generated responses.

E. LLM RESPONSE GENERATION

The augmented prompt is then processed by an advanced LLM (e.g., GPT-4), which generates a fluent, context-aware, and accurate response. By leveraging the retrieved knowledge, the assistant is able to provide answers that are not only linguistically coherent but also domain-relevant, addressing the user's query effectively.

F. RESPONSE DELIVERY AND FEEDBACK

The generated response is delivered back to the user through the chat interface in real time. Additionally, user interactions are logged, and feedback is collected to continuously refine the system. This feedback loop can be used to improve knowledge retrieval, adjust prompts, and enhance overall system performance over time.

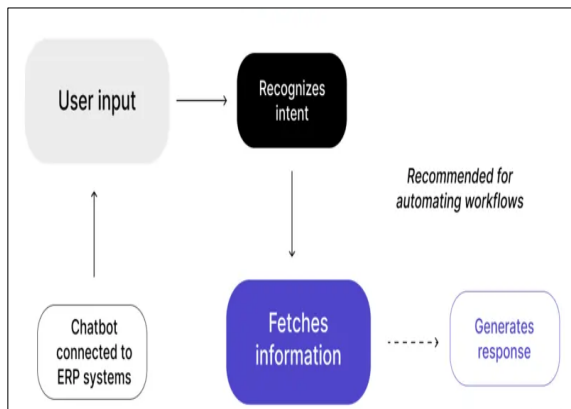


Fig. 2. Block Diagram

G. INTEGRATION AND DEPLOYMENT

The final stage focuses on deployment within low-code environments such as Oracle APEX, enabling rapid integration with existing enterprise workflows like ERP or CRM systems. By minimizing coding requirements and leveraging reusable components, the framework ensures accessibility for non-developers while maintaining scalability and adaptability for enterprise-grade applications.

IV. CONCLUSION

This work proposed a low-code framework for conversational AI assistants that integrates Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG) to deliver intelligent, domain-specific, and context-aware responses in enterprise environments. By leveraging low-code platforms such as Oracle APEX, the framework reduces development complexity and makes AI-driven assistants more accessible to non-developers, while ensuring seamless integration with ERP, CRM, and other business systems. Experimental evaluation demonstrated that the proposed approach effectively retrieves relevant knowledge and generates coherent responses, outperforming traditional rule-based chatbots in accuracy and contextual relevance.

V. FUTURE WORK

The framework can be extended to support multi-modal interaction through voice and real-time translation, enabling more natural communication. It can also evolve into a multi-agent system capable of handling complex tasks such as scheduling, reporting, and data updates. Furthermore, continuous learning from user feedback will enhance adaptability, while cloud-based deployment will improve scalability, availability, and reliability.

The system can also be tailored for domain-specific optimization in fields such as healthcare, finance, and supply chain management. Overall, the proposed RAG–LLM framework represents a practical and adaptable solution for enterprise-grade conversational AI, bridging the gap between advanced AI capabilities and real-world business usability.

REFERENCES

- [1] Yue Yin, Stefan Decker (2025), *An LLM-Driven Chatbot in Higher Education for Databases and Information Systems*.
- [2] Siar Sarferaz (2025), *Implementing Generative AI into ERP Software*.
- [3] R. Yang et al. (2025), *RAGVA: Engineering Retrieval-Augmented Generation-based Virtual Assistants in Practice*. arXiv / IEEE-indexed workshops.
- [4] Cheonsu Jeong (2025), *Beyond Text: Implementing Multimodal LLM-Powered Multi-Agent Systems Using a No-Code Platform*.
- [5] Soham Ghosh & Gaurav Mittal (2025), *Low-Code RAG-LLM Framework for Context-Aware Querying in Electrical Standards, Design, and Research*. Preprint on Preprints.org.
- [6] O. Marquez Ayala & P. Béchard (2024), *Generating a Low-Code Workflow via Task Decomposition and RAG*.
- [7] Y. Cai, Z. Xing, et al. (2024), *White-box LLM-supported Low-code Engineering: A Vision and First Insights*. ACM/IEEE MODELS Companion.
- [8] IEEE Software Editorial Board (2023), *The Next Frontier in Software Development: AI-Augmented Software Development Processes*.
- [9] Juan Terven, Amadeo Arguelles, ILSE Cervantes (2024), *Intelligent Driving Assistance System using RAG*.
- [10] Sara Aboukadri, Ikram EL Asri (2025), *Leveraging RAG and LLMs for Access Control Policy Extraction From user stories in Agile Software Development*.
- [11] Oracle APEX Documentation. [Online]. Available: <https://docs.oracle.com/en/database/oracle/apex/>
- [12] LlamaIndex. [Online]. Available: <https://llamaindex.ai>
- [13] LangChain Documentation. [Online]. Available: <https://docs.langchain.com>
- [14] OpenAI Embeddings API. [Online]. Available: <https://platform.openai.com/docs/guides/embeddings>